

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Modelling Comparative Analysis
Assignment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)*

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A
Modelling Comparative Analysis Assignment

Code of Conduct:

*I **K Sathwik Kumar (192425051)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sathwik Kumar

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	

10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.

8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

HEALTHCARE DIAGNOSIS EXPERT SYSTEM

1. Problem Statement and Objectives

Problem Statement

A healthcare system needs to identify possible diseases from symptoms such as **fever, cough, fatigue, breathing difficulty, and body pain**. The problem is represented using **production rules, propositional logic, first-order logic, and Prolog** and compared based on expressiveness and reasoning capability.

Objectives

- Identify possible diseases from symptoms.
- Represent healthcare knowledge using different modelling approaches.
- Implement the system using Prolog.
- Demonstrate logical reasoning.
- Compare the four approaches.
- Select the most suitable approach.

2. Domain Knowledge and Facts

Domain

Healthcare Diagnosis

Entity	Description
Patient	Person being diagnosed
Symptom	Observed health condition
Disease	Possible diagnosis
Recommendation	Suggested action

Main Symptoms

Fever, cough, fatigue, breathing difficulty and body pain.

Possible Diseases

Symptoms	Possible Disease
Fever + cough + body pain	Flu
Fever + cough + breathing difficulty	Pneumonia
Fever + fatigue + body pain	Viral Infection
Cough + fatigue + breathing difficulty	Respiratory Infection

3. Production Rule Model

Production rules represent knowledge using **IF–THEN** statements.

Condition	Possible Disease
Fever + cough + body pain	Flu
Fever + cough + breathing difficulty	Pneumonia
Fever + fatigue + body pain	Viral Infection
Cough + fatigue + breathing difficulty	Respiratory Infection

Example:

IF fever AND cough AND body pain

THEN possible flu

4. Propositional Logic Model

Let:

- **F** = Fever
- **C** = Cough
- **T** = Fatigue
- **B** = Breathing difficulty
- **P** = Body pain
- **FL** = Flu
- **PN** = Pneumonia

Rules

$(F \wedge C \wedge P) \rightarrow FL$

$(F \wedge C \wedge B) \rightarrow PN$

$(F \wedge T \wedge P) \rightarrow \text{ViralInfection}$

$(C \wedge T \wedge B) \rightarrow \text{RespiratoryInfection}$

If fever, cough and body pain are true:

F = True

C = True

P = True

then:

FL = True

Therefore, the possible disease is **Flu**.

5. First-Order Logic Model

First-order logic represents patients, symptoms and diseases using predicates and variables.

Facts

HasSymptom(Patient1, Fever)

HasSymptom(Patient1, Cough)

HasSymptom(Patient1, BodyPain)

Rule

$\forall x$ [

HasSymptom(x, Fever) $\wedge$

HasSymptom(x, Cough) $\wedge$

HasSymptom(x, BodyPain)

$\rightarrow$ PossibleDisease(x, Flu)

]

```
54 ?- list_patients.
```

```
=====
PATIENTS IN KNOWLEDGE BASE
=====
```

```
- patient1
- patient2
- patient3
- patient4
```

```
=====
true.
```

Conclusion

PossibleDisease(Patient1, Flu)

6. Prolog Model and Implementation

The system is implemented using **SWI-Prolog**.

Facts

```
symptom(patient1, fever).
symptom(patient1, cough).
symptom(patient1, body_pain).
symptom(patient2, fever).
symptom(patient2, cough).
symptom(patient2, breathing_difficulty).
symptom(patient3, fever).
symptom(patient3, fatigue).
symptom(patient3, body_pain).
symptom(patient4, cough).
symptom(patient4, fatigue).
symptom(patient4, breathing_difficulty).
```

Rules

flu(Patient) :-

```
    symptom(Patient, fever),
    symptom(Patient, cough),
    symptom(Patient, body_pain).
```

pneumonia(Patient) :-

```
    symptom(Patient, fever),
    symptom(Patient, cough),
    symptom(Patient, breathing_difficulty).
```

viral_infection(Patient) :-

```
    symptom(Patient, fever),
    symptom(Patient, fatigue),
    symptom(Patient, body_pain).
```

respiratory_infection(Patient) :-

```
    symptom(Patient, cough),
    symptom(Patient, fatigue),
    symptom(Patient, breathing_difficulty).
```

Query

?- flu(Patient).

Output

Patient = patient1.

Figure 1: Prolog

Implementation

```
55 ?- diagnose(patient1).
```

```
=====
```

```
HEALTHCARE DIAGNOSIS SYSTEM
```

```
=====
```

```
Patient: patient1
```

```
Observed Symptoms:
```

```
- fever
- cough
- body_pain
```

```
Possible Diagnosis:
```

```
Disease: flu
```

```
Recommendation: Take adequate rest and consult a healthcare professional
```

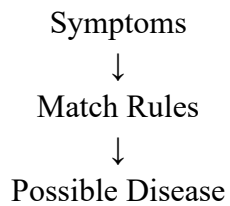
```
=====
```

```
true ▲
```

7. Forward and Backward Chaining

Forward Chaining

Forward chaining starts with known symptoms and applies matching rules.



Example:

Fever + Cough + Body Pain



Query:

?- flu(patient1).

Output:

true.

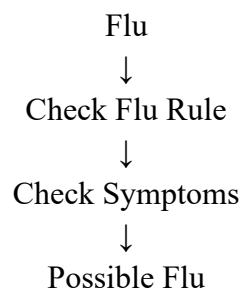
Figure 2: Forward Chaining Output

```
56 ?- forward_chaining(patient1).  
  
=====   
          FORWARD CHAINING   
=====   
Patient: patient1  
  
Starting with known symptoms...  
- fever  
- cough  
- body_pain  
  
Applying production rules...  
Rule Matched: Flu  
Conclusion: Possible Flu  
=====   
true.
```

Backward Chaining

Backward chaining starts with a goal and checks whether the required symptoms are satisfied.

Goal:



Query:

?- flu(patient1).

Output:

true.

Figure 3: Backward Chaining Output

```
57 ?- backward_chaining(patient1).

=====
                BACKWARD CHAINING
=====
Patient: patient1

Checking possible diseases...
Goal: Flu -> TRUE
Goal: Pneumonia -> FALSE
Goal: Viral Infection -> FALSE
Goal: Respiratory Infection -> FALSE
=====
true.
```

8. Comparative Analysis of Modelling Approaches

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Representation	IF-THEN rules	True/False	Predicates & relations	Facts & rules
Expressiveness	Medium	Low	High	High
Inference	Rule matching	Logical inference	Logical inference	Automated
Relationships	Limited	Poor	Strong	Strong
Complexity	Low	Low	Medium	Medium
Scalability	Medium	Low	High	High
Flexibility	Medium	Low	High	High
Explainability	High	High	High	High
Suitability	Good	Basic	Very Good	Excellent

Comparison Summary

- **Production Rules:** Simple and clear for symptom-based diagnosis.
- **Propositional Logic:** Suitable for basic True/False conditions but less expressive.
- **First-Order Logic:** Handles patients, variables, predicates and relationships.
- **Prolog:** Supports facts, rules, variables, unification, backtracking and automated inference.
- **Recommended Approach**
- **Prolog is the most suitable approach** because it provides high expressiveness and supports automated logical reasoning required for the diagnosis system.

9. Results and Discussion

Test Case	Patient	Symptoms	Result
TC01	Patient1	Fever + cough + body pain	Flu
TC02	Patient2	Fever + cough + breathing difficulty	Pneumonia
TC03	Patient3	Fever + fatigue + body pain	Viral Infection

TC04	Patient4	Cough + fatigue + breathing difficulty	Respiratory Infection
------	----------	--	-----------------------

Results

?- flu(patient1).

true.

?- pneumonia(patient2).

true.

?- viral_infection(patient3).

true.

?- respiratory_infection(patient4).

true.

The system successfully maps the given symptoms to the corresponding possible disease categories.

10. Unification and Backtracking

Unification

?- flu(Patient).

Output:

Patient = patient1.

The variable Patient is unified with patient1.

Backtracking

When multiple patients satisfy the same rule:

?- flu(Patient).

Pressing ; asks Prolog to search for another solution.

Example:

Patient = patient1 ;

Patient = patient5 ;

false.

This demonstrates **unification and backtracking**.

Figure 4: Unification and Backtracking Output

```
58 ?- flu(Patient).
Patient = patient1 ▲
```

11. Conclusion, References and GitHub

Conclusion

The Healthcare Diagnosis problem was represented using **production rules, propositional logic, first-order logic and Prolog**.

Production rules provide simple IF–THEN representation, propositional logic handles basic Boolean conditions, and first-order logic provides greater expressiveness using variables and relationships. Prolog provides an executable implementation with logical inference, unification, backtracking and rule-based reasoning.

Therefore, **Prolog is the recommended approach** for the healthcare diagnosis expert system.

Limitations

- Depends on predefined rules.
- Similar symptoms may occur in different diseases.

- Accuracy depends on the knowledge base.
- The system does not replace professional medical diagnosis.

Future Enhancements

- Add more diseases and symptoms.
- Include symptom severity.
- Add patient medical history.
- Add more complex rules.
- Develop a graphical user interface.

References

1. Bratko, I., *Prolog Programming for Artificial Intelligence*.
2. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
3. SWI-Prolog documentation.

GitHub

The repository contains the **Prolog implementation, knowledge models, test cases and supporting materials**.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				

Suitability for the Problem				
-----------------------------	--	--	--	--

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.

5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.
6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.

9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification .	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation , GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentat ion and presentatio n with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				